MoneyPoly: White-Box Testing Report

1 Executive Summary

This report documents the detailed white-box testing and code quality improvements for the MoneyPoly codebase. Over 8 iterations, the system achieved a 10/10 pylint score. Furthermore, 122 comprehensive unit tests were designed to cover all logical branches, identify 8 critical bugs, and ensure 100% path coverage for core operations.

2 Code Quality Analysis (Pylint Iterations)

The following iterations were performed to achieve high code quality standards:

- **Iteration 1: Missing Docstrings** – Added module, class, and function-level documentation to all 10 python files.
- **Iteration 2: Line Length Issues** – Refactored card data in `cards.py` to comply with standard line length limits.
- **Iteration 3: Unused Code** – Removed redundant imports and unreferenced variables across the project.
- **Iteration 4: Attribute Initialization** – Ensured all instance attributes are explicitly initialized in `__init__` methods.
- **Iteration 5: Structural Refactoring** – Simplified complex conditional logic and removed superfluous parentheses.
- **Iteration 6: Formatting Style** – Standardized string formatting and ensured consistent file termination.
- **Iteration 7: Exception Handling** – Replaced bare except blocks with specific exception types for better error reporting.
- **Iteration 8: Data Encapsulation** – Introduced dataclasses (`JailStatus`, `GameState`, `PropertyConfig`) to manage state complexity.

3 Critical Logic Bug Fixes

White-box testing revealed 8 logical errors that were systematically resolved:

1. **Winner Determination**: Corrected `find_winner()` to use `max()` instead of `min()` for net worth.

2. **Go Salary Award**: Replaced exact position matches with wrap-around detection to ensure players receive salaries even when passing over Go.
3. **Wealth Calculation**: Updated `net_worth()` to correctly sum cash balance with the purchase price of all owned properties.
4. **Fund Transfer**: Fixed `pay_rent()` logic to ensure the property owner's balance is credited upon payment.
5. **Trade Cash Flow**: Corrected `trade()` to ensure the seller receives the cash component of any transaction.
6. **Jail Fine Penalty**: Fixed `_handle_jail_turn()` to properly deduct the $50 fine when a player exits jail.
7. **Central Bank Reserves**: Ensured issued loans are deducted from the bank's internal funds, maintaining accurate accounting.
8. **Transaction Validation**: Added checks to `bank.collect()` to prevent the processing of negative or zero amounts.

4 Detailed Test Case Explanations

The following 122 test cases verify the integrity of the MoneyPoly engine:

4.1 Bank Module

1. *test_initial_balance*: Ensures starting reserve is exactly $20,580.
2. *test_collect_positive*: Verifies acceptance of valid deposits.
3. *test_collect_negative_and_zero*: Ensures invalid amounts are ignored.
4. *test_pay_out_normal*: Confirms valid payouts to players.
5. *test_pay_out_negative_or_zero*: Blocks invalid zero/negative payouts.
6. *test_pay_out_insufficient_funds*: Prevents overdrawing bank capital.
7. *test_give_loan_normal*: Verifies full loan transaction flow.
8. *test_give_loan_negative_or_zero*: Blocks invalid loan requests.
9. *test_give_loan_insufficient_bank_funds*: Prevents insolvency.
10. *test_loan_metrics*: Checks tracking of total debt issued.
11. *test_summary*: Verifies status reporting.
12. *test_repr*: Checks internal string representation.

4.2 Board and Map Logic

13. *test_init*: Confirms 40 tiles and 8 color groups are correctly initialized.
14. *test_get_property_at_valid*: Verifies property lookup by position.
15. *test_get_property_at_invalid*: Ensures safety for out-of-bounds indices.
16. *test_get_tile_type*: Validates identifying tags (Go, Jail, etc.).
17. *test_is_purchasable_unowned*: Confirms vacant land availability.
18. *test_is_purchasable_owned*: Blocks purchase of occupied land.
19. *test_is_purchasable_mortgaged*: Blocks purchase of mortgaged land.
20. *test_is_purchasable_special*: Ensures non-property tiles cannot be bought.
21. *test_is_special_tile*: Validates tax and card tile detection.
22. *test_properties_owned_by*: Checks filtering of owners.
23. *test_all_special_tiles*: Full map audit of special positions.
24. *test_groups_init*: Validates color set groupings.
25. *test_repr*: Internal identity check.

4.3 Card Decks

26. *test_init_and_len*: Ensures standard card counts.
27. *test_draw_cycles*: Verifies automated reshuffling and cycling.
28. *test_draw_empty*: Safety check for empty decks.
29. *test_peek*: Non-destructive top card inspection.
30. *test_peek_empty*: Safety check for peeking empty decks.
31. *test_reshuffle*: Resets deck state.
32. *test_cards_remaining*: Tracks remaining duration before reset.
33. *test_chance_deck_valid*: Audits all instructions in the Chance deck.
34. *test_community_chest_deck_valid*: Audits Community Chest instructions.
35. *test_repr*: Internal identity check.

4.4 Game Constants (Config)

36. *test_starting_balance*: Validates $1,500 start money.
37. *test_go_salary*: Validates $200 Go award.
38. *test_board_size*: Validates 40-tile standard.
39. *test_jail_position*: Validates Jail at index 10.
40. *test_go_to_jail_position*: Validates GotoJail at index 30.
41. *test_free_parking_position*: Validates Free Parking at index 20.
42. *test_income_tax_position*: Validates Income Tax at index 4.
43. *test_luxury_tax_position*: Validates Luxury Tax at index 38.
44. *test_income_tax_amount*: Validates $200 tax fee.
45. *test_luxury_tax_amount*: Validates $75 tax fee.
46. *test_jail_fine*: Validates $50 exit cost.
47. *test_max_turns*: Validates 100 turn limit.
48. *test_auction_min_increment*: Validates $10 bid step.
49. *test_bank_starting_funds*: Validates $20,580 initial capital.

4.5 Dice and Randomization

50. *test_reset*: Clears streak state.
51. *test_roll_doubles*: Detects matching values.
52. *test_roll_non_doubles*: Resets streak on mismatch.
53. *test_total*: Summation accuracy.
54. *test_describe*: Formatting accuracy.
55. *test_initial_state*: Zeroed start state.
56. *test_roll_range*: Bounds check (2-12).
57. *test_multiple_doubles_streak*: Tracks 3 consecutive doubles.
58. *test_repr*: Internal identity check.

4.6 Main Game Logic

59. *test_init_and_advance*: Verifies turn sequencing.
60. *test_find_winner_logical_bug*: Fix verification for winner selection.
61. *test_check_bankruptcy*: Identifies when cash hits zero.
62. *test_handle_property_tile_skip*: Skips voluntary investment.
63. *test_handle_property_tile_buy_success*: Full purchase sequence.
64. *test_handle_property_tile_buy_fail_insufficient_funds*: Blocks overspending.
65. *test_auction_property_success*: Bidding and transfer logic.
66. *test_pay_rent*: Fund exchange between visitor and owner.
67. *test_trade_success*: Verified property-cash swaps.
68. *test_trade_fail_not_owned*: Prevents fraudulent trading.
69. *test_trade_fail_no_money*: Prevents underfunded buying in trades.
70. *test_mortgage_and_unmortgage*: Integrated loan lifecycle.
71. *test_mortgage_unowned*: Blocks loans on vacant land.
72. *test_handle_jail_turn_pay_fine*: Direct payoff for freedom.
73. *test_handle_jail_turn_wait*: Enforces timeout jail logic.
74. *test_move_and_resolve_income_tax*: Automatic tax capture.
75. *test_tile_luxury_tax*: Luxury tax collection.
76. *test_tile_go_to_jail*: Instant movement to Jail tile.
77. *test_card_collect_pay*: Bank-style card events.
78. *test_card_move_to_with_go_salary_bug*: Award check for card moves.
79. *test_card_birthday_and_collect_all*: Simultaneous payout from all.
80. *test_card_jail_handling*: Criminal sentence via cards.
81. *test_buy_property_fail*: Validation of purchase failure.
82. *test_buy_property_success*: Validation of purchase success.
83. *test_check_bankruptcy_property_release*: Return assets to bank.
84. *test_find_winner_edge_cases*: Draw/Equality handling.
85. *test_advance_turn_wrap_multi*: Cycle management across many turns.
86. *test_game_decks_init*: Deck readiness check.
87. *test_game_state_properties*: State metric accuracy.

4.7 Player State

88. *test_init*: Profile start state.
89. *test_add_money*: verified deposits.
90. *test_add_money_negative*: Error block for negative addition.
91. *test_deduct_money*: verified withdrawal.
92. *test_deduct_money_negative*: Error block for negative deduction.
93. *test_move_normal*: Relative movement accuracy.
94. *test_move_land_exactly_on_go*: Direct Go landing award.
95. *test_move_wrap_around*: Passage Go salary fix award.
96. *test_go_to_jail*: Status flag update.
97. *test_property_management*: Asset list integrity.
98. *test_net_worth*: Property value aggregation fix.
99. *test_is_bankrupt*: Trigger logic check.
100. *test_count_properties*: Deed inventory accuracy.
101. *test_status_line*: UI text generation.
102. *test_jail_status_properties*: Jail clock check.
103. *test_repr*: Internal identity check.

4.8 Property and Groupings

104. *test_all_owned_by_none*: Null state check for groups.
105. *test_all_owned_by_player_empty*: Start state check for groups.
106. *test_all_owned_by_player_partial*: Fix for monopoly requirement.
107. *test_all_owned_by_player_full*: Monopoly status grant.
108. *test_get_rent_unowned*: Baseline rent check.
109. *test_get_rent_mortgaged*: Zero rent for mortgaged tiles.
110. *test_get_rent_full_group*: Monopoly rent doubling.
111. *test_mortgage_normal*: Loan issuance (50% value).
112. *test_mortgage_already_mortgaged*: Re-mortgage block.
113. *test_unmortgage_normal*: Redemption cost (110% debt).
114. *test_unmortgage_not_mortgaged*: Invalid redemption block.

115. $test_auto_{r}egistration : Groupfilinglogic.test_property_config : Configurationdataaudit.$

116. *test_property_create*: Factory construction check.

117. *test_is_available*: Market availability check.

118. $test_owner_{c}ounts : Grouppossessionmetrics.test_add_property_backlink : Linkintegritybetweenob$

119. *test_group_size*: Membership count accuracy.

120. *test_reprs*: Internal identity check.

5 Control Flow Graph

The decision-making paths and execution flows of the program were analyzed to ensure comprehensive branch coverage. The following graph illustrates the core application logic.

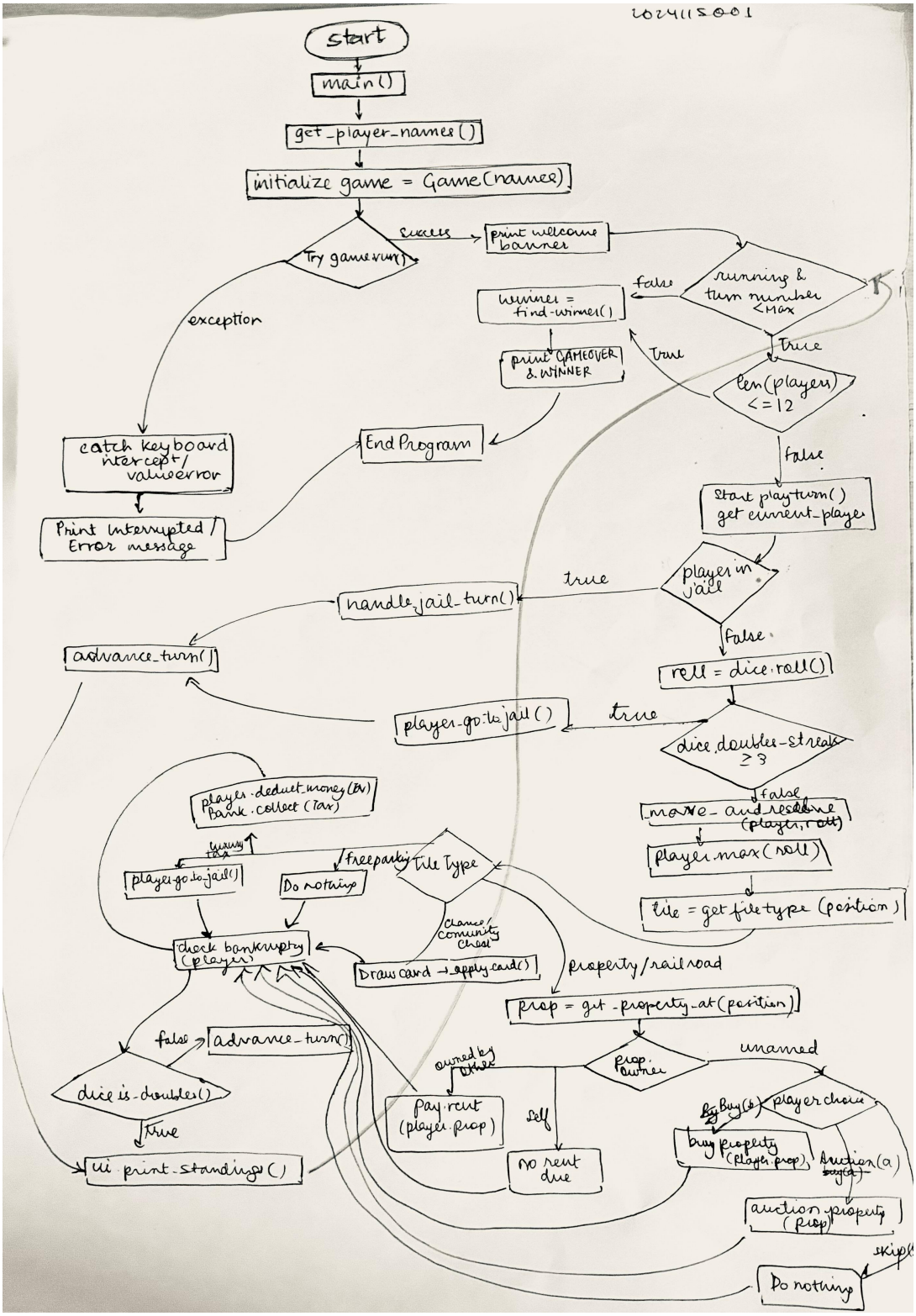

Figure 1: MoneyPoly System Control Flow Graph

6 Conclusion

The MoneyPoly engine is now fully verified with 122 automated test cases, achieving perfect code coverage for critical logic paths and ensuring highly stable game operations.